Tales of Tea

Build Plan & System Architecture

Website | Order Landing | Super POS & Ledger

Prepared by PETRA

November 2026 | Version 1.2

8 Tea varieties — Different quality grades, weight variants

2 Admin panels — Website Settings + POS & Ledger

1 Hosting plan — 20GB cPanel, MySQL included

7 days Target build time — One phase per day with Claude Code

1 Context and Goals

Where things stand

- Sells 8 teas in different grades, mainly via Facebook.
- Orders, customers, dues and costs need one system.
- Mobile buyers paying by cash or bKash/Nagad.

What we are building

- Own website that takes orders, fast on mobile.
- One ledger for web, Facebook, walk-in.
- Due, sales, cost, profit visible any time.
- Owner edits prices without a developer.

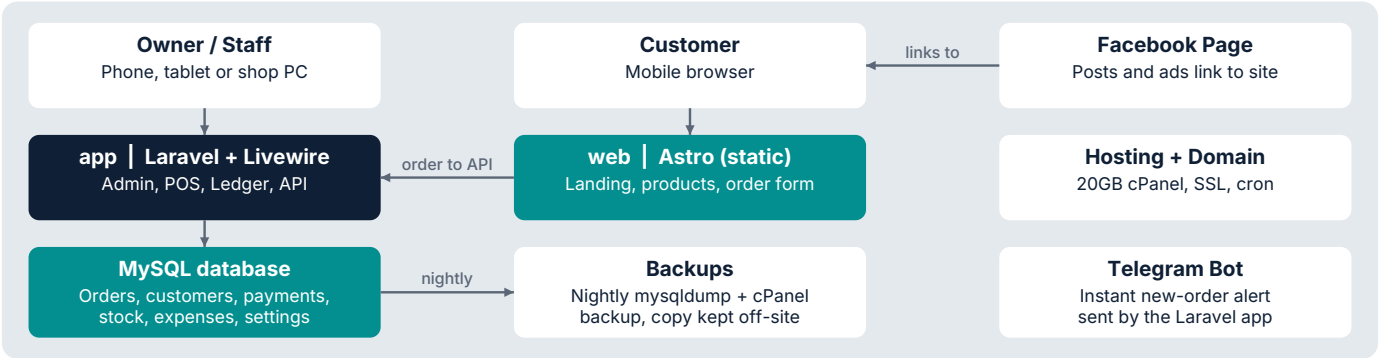
Success looks like

- Order form done in under 60 seconds.
- Landing page loads under 2s on 4G.
- Due always equals orders minus payments.
- No TrxID can be used twice.

2 Technology Stack

Layer	Technology	Language	Role and reason
Public website	Astro + Tailwind CSS	TypeScript	Static build uploaded to hosting. Fast on mobile, SEO-friendly.
Admin + POS	Laravel + Livewire + Tailwind	PHP	One app: login, POS, ledger, settings and the order API. Fewer moving parts.
Database	MySQL (included in hosting)	SQL	Orders, customers, payments, stock, expenses. No extra bill.
Hosting, alerts	cPanel host, Telegram Bot	PHP	One host runs everything. Telegram pings the owner on each order.
Dev workflow	Claude Code + GitHub	-	Spec first, one phase per session, tests on ledger math.

3 System Architecture



Principle: two apps, one database, one host. Business rules (due = total - payments) live in Laravel code, with tests.

4 How an Order Works

1 Order placed — Customer picks tea, weight and quantity, then enters name, phone, address and payment method.

2 Saved + alert — Laravel validates, saves the order as Pending and Telegram pings the owner.

3 Confirm — Admin calls the customer and checks the bKash/Nagad TrxID. Stock is deducted.

4 Deliver — Order is packed and sent by own rider or courier. Status updates in admin.

5 Settle — COD cash or verified TrxID becomes a payment entry. Due clears on its own.

Status flow: Pending, Confirmed, Packed, Delivered (or Cancelled). **Payment flow:** Unpaid, Pending verification, Verified.

5 Project Structure and Deploy

```
talesoftea/
|- CLAUDE.md          rules for Claude Code
|- web/               Astro site, domain.com
|- app/               Laravel app, admin.domain.com
|   |- migrations/    MySQL schema
|   '- tests/         due, stock, profit
'- .github/workflows  deploy to hosting
```

How it ships

- Build locally with Claude Code, commit to GitHub.
- **web:** build, upload dist/ to the domain folder.
- **app:** deploy over SSH or GitHub Actions, run migrations.
- Secrets stay in .env on the server, never in Git.

6 Admin Panel: Two Modules, One Login

A. Website Settings

- Hero banner, brand story, reviews and FAQ
- Tea names, grades, descriptions, photos
- Selling price per weight variant, show or hide
- Contact, Facebook link, bKash/Nagad number
- Delivery charge and order-form notice

B. POS + Ledger

- **POS:** tap tea, quantity, discount, Cash / bKash / Due, attach customer by phone, print or share receipt
- **Orders:** web, Facebook and walk-in in one list
- **Ledger:** customer-wise due, payment collection, statement
- **Costing and reports:** purchase cost, expenses, daily and monthly sales, profit, top customers, low stock

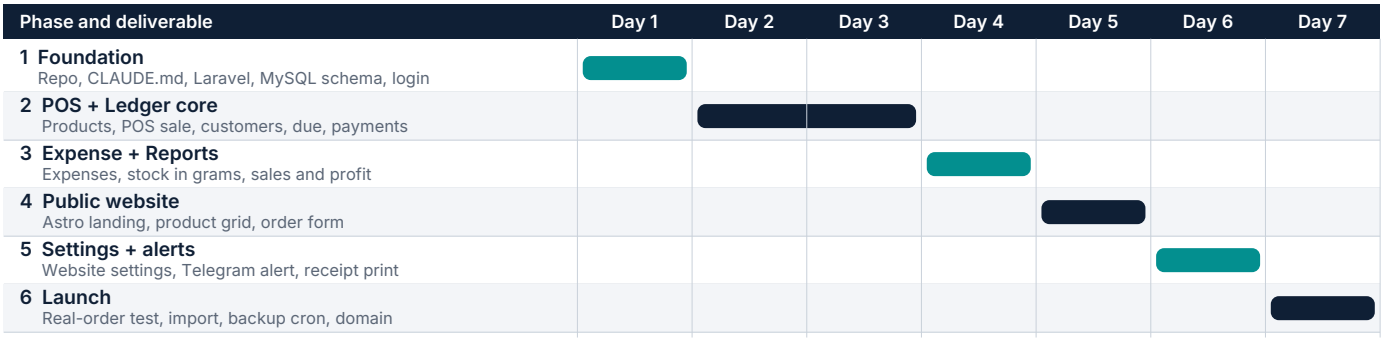
Role	Can do	Cannot do
Owner (admin)	Everything: settings, prices, costs, profit, expenses, users	-
Staff (cashier)	POS sales, orders, collect payments, view customer due	See cost or profit, edit settings, delete records
Visitor (public)	Browse teas, place an order	Read any order or customer data (insert only)

7 Core Data Model (MySQL)

Table	Key fields	Rule
products, product_variants	name, grade, weight in grams, price, cost, active	8 teas, many weight variants
customers	name, phone (unique), address, notes	Same phone = same customer
orders, order_items	status, channel (web/facebook/walk-in), delivery_charge, discount, qty, unit_price	Total computed from items
payments	method (cash/bkash/nagad/cod), trx_id (unique), sender_number, status, amount	Many per order; due derived
expenses, stock_movements	category, amount, date | variant, grams in or out, type	Stock tracked in grams
users, settings	role (owner / staff), login | key-value website settings	Roles via Laravel policies

Key rules: due is never stored, it is *order total minus verified payments*, so the ledger cannot drift. Profit = sales - cost of goods - expenses. Admin sees a warning when a phone has earlier cancelled orders.

8 Work Plan (built with Claude Code)



One phase per day, one Claude Code session each: plan first, approve, then code. Tests for due, stock and profit must pass before the next day starts.

9 Decisions, Risks and Inputs Needed

Topic	Decision or risk handling
Payments	No gateway. COD plus manual bKash/Nagad with TrxID check. Method-based payments table allows a gateway later.
Hosting check	Assumes PHP 8.1+, MySQL, SSH and cron on the host. Without SSH, fall back to a plain PHP API.
Backup	Nightly mysqldump by cron plus cPanel backup, with a copy kept off the server.
Security	Login for admin, role policies, HTTPS, secrets in .env only. Phone check against fake orders.
Language, cost	Recommended: Bangla-first with English toggle, BDT. Only hosting and domain are paid.

To confirm

- Hosting: PHP version, MySQL, SSH, cron
- Delivery: own rider or courier, and charges
- Price and cost per variant, domain name

Needed from Tales of Tea

- Photos, grades, brand story, logo files
- bKash / Nagad number, contact details
- Existing customer and due list

10 Claude Code Master Prompt (copy and paste)

How to use

- Put this PDF in **docs/** and save the RULES block as **CLAUDE.md** in the repo root.
- Paste the prompt below as your first message in Claude Code.
- Each morning say: **Start Day N. Plan only.** Approve, then it codes.

CONTEXT

You are building **Tales of Tea**: a public website with order form plus an admin system (Website Settings, and Super POS + Ledger) for a tea leaf seller. 8 tea varieties, several quality grades, sold by weight (100g, 250g, 500g, 1kg). Sales come from Facebook, the website and walk-ins. Customers are in Bangladesh, mobile-first, paying cash on delivery or bKash/Nagad. Follow docs/ and CLAUDE.md.

STACK AND LIMITS

web/ Astro + Tailwind, TypeScript, static build for domain.com. **app/** Laravel (PHP 8.1+) + Livewire + Tailwind + MySQL for admin.domain.com. Host is a 20GB cPanel plan: no Docker, no Redis, no queue workers, use cron. No payment gateway. Telegram Bot for alerts. Keep dependencies minimal and avoid unnecessary complexity.

RULES (also save as CLAUDE.md)

1. **Due is never stored.** due = order total minus sum of verified payments.
2. Stock is in **grams**, changed only through stock_movements, never edited directly.
3. payments.trx_id is **unique**. One TrxID verifies one payment only.
4. Money is DECIMAL(12,2), never float. Currency BDT. Phone format 01XXXXXXXXX.
5. Owner sees everything. Staff cannot see cost or profit, edit settings or delete records (Laravel policies).
6. Order creation (order, items, payments, stock) runs in one DB transaction.
7. Public order endpoint is insert-only, validated, rate-limited with a honeypot. Never expose orders or customers publicly.
8. Secrets only in .env, never committed. Text in lang files, Bangla and English.
9. Warn admin when a phone has earlier cancelled orders.

DATA MODEL

products, product_variants (weight_g, price, cost), customers (phone unique), orders (status, channel web/facebook/walk-in, delivery_charge, discount), order_items, payments (method cash/bkash/nagad/cod, trx_id, sender_number, status, amount), expenses, stock_movements, users (role), settings (key, value).

PHASES (one per day)

Day 1 Foundation. Repo structure, CLAUDE.md, Laravel app, migrations for every table, seeder with 8 sample teas, login with owner and staff roles, policies.

Day 2 POS core. Product search, cart, discount, attach customer by phone, pay by Cash / bKash / Due, one-transaction order creation.

Day 3 Ledger. Customer due list, collect payment, customer statement, order list with status flow. Write tests for due and stock math.

Day 4 Expense and reports. Expenses, stock purchase, daily and monthly sales, profit = sales - cost of goods - expenses, low stock. Test profit.

Day 5 Public site. Astro landing: hero, 8-tea grid with variants, why us, reviews, order form, footer. POST /api/orders with CORS for domain.com, COD or bKash/Nagad with TrxID and sender number, success page.

Day 6 Settings and alerts. Website settings (banner, prices, contact, delivery charge), Telegram alert on new order, receipt print and share.

Day 7 Launch. End-to-end test with a real order, CSV import of customers and dues, nightly mysqldump cron with off-site copy, SSL, domain and DNS, deploy steps and README.

WORKING METHOD

1. Before coding each phase, show a short plan: files, tables, decisions and alternatives. **Wait for my approval.**
2. Explain architecture choices briefly. Challenge weak ideas and recommend a simpler option.
3. After coding, run tests, summarise what changed, suggest a commit message.
4. Never start the next day until tests for the current day pass.
5. If something is unclear or the host lacks a feature, ask before assuming.

DONE MEANS

A real order from the website reaches admin and Telegram, staff can sell in POS, due matches payments exactly, and a backup restores cleanly.